# Software Maintenance Report - Copy/Paste in JHotDraw

## Maintaining the Basic Editing Copy/Paste Feature

[Fill in full name]

11 June 2026

## Table of Contents

## Report Information

| Field | Value |
|---|---|
| Full name | [Fill in full name] |
| Student exam number | [Fill in exam number] |
| Student email | [Fill in student email] |
| Course number | SB5-MAI Software Maintenance, 5 ECTS |
| Lecturer | [Fill in lecturer name and email] |
| Repository | `https://github.com/JakubPotocky/JHotDraw` |
| Inspected branch | `copy-paste-basic-editing-labs` |
| Inspected commit | `b4eea56487ea289d6c1855d9ba32150e16b31366` |
| Local checkout | `/home/bob/Desktop/university/softwareMaintenance/JHotDraw` |
| Number of pages | Update after final PDF export |

## Abstract

This report is part of the 5 ECTS Software Maintenance course. The objective of the course is to maintain an existing software repository by locating a feature in unfamiliar code, analysing the impact of a change, removing code smells,

improving the internal structure, and verifying that the system still behaves correctly.

The case study is JHotDraw, a Java/Swing framework for structured two-dimensional drawing editors. The selected work area is the Basic Editing Copy/Paste feature. Copy/Paste already existed in JHotDraw, so the work documented here is not a new feature implementation. It is a preventive maintenance change: understand the existing implementation, remove duplicated paste logic, clean small documentation/code-quality smells, and add automated tests that preserve the current user-visible behaviour.

On branch `copy-paste-basic-editing-labs`, the main production change is in `DefaultDrawingViewTransferHandler`. The paste import path was refactored with Extract Method by introducing `importTransferData(...)` and `firePasteUndoableEdit(...)`. A dead private method was removed, comments were improved, and `CompositeTransferable` received small documentation and immutability cleanup. Verification was added through 13 JUnit 4 tests, including 5 BDD-style Given-When-Then scenarios. Local Maven verification was run for both the sample module and the full reactor, and both completed with `BUILD SUCCESS`.

This report follows the course report template: Introduction, Initiation, Concept Location, Impact Analysis, Prefactoring, Actualization, Postfactoring, Verification, Conclusion, Discussion, References, and Appendix. It also records limits honestly: no remote GitHub Actions result for this feature branch was available from the local evidence, and the workflow currently triggers on `develop` push/pull-request events rather than arbitrary feature-branch pushes.

## Introduction

JHotDraw is a Java framework for building structured drawing editors. It is a useful software-maintenance case study because it is a real multi-module Maven project with legacy Swing code, clipboard/data-transfer infrastructure, design patterns, input/output format strategies, and accumulated technical debt. The maintainer must locate the relevant concept before changing anything, because the user-visible feature is spread across several layers rather than implemented in one obvious method.

The selected feature work area is Basic Editing Copy/Paste. A user expects to select a figure, copy it, paste it back into the drawing, and continue editing without manually recreating the same shape. This is a core editor feature, but its implementation is composite:

| Layer | Copy/Paste responsibility |
|---|---|
| Actions | `CopyAction`, `CutAction`, and `PasteAction` expose user commands. |
| Clipboard | `ClipboardUtil`, `AWTClipboard`, `OSXClipboard`, and `CompositeTransferable` handle clipboard access and transferable flavours. |
| Swing transfer | `TransferHandler` is the Swing mechanism used by copy, cut, paste, and drag/drop. |
| Drawing view | `DefaultDrawingView` owns selection state and installs `DefaultDrawingViewTransferHandler`. |
| Transfer handler | `DefaultDrawingViewTransferHandler` exports selected figures and imports supported clipboard data. |
| Model | `Drawing` and `Figure` represent the drawing and copied/pasted objects. |
| Formats | `InputFormat` and `OutputFormat` define how figure data, image data, and text data are read or written. |

The maintenance objective was:

1. Locate the existing Copy/Paste implementation.
2. Analyse the impact of changing it.
3. Remove duplicated paste logic without changing user-visible behaviour.
4. Preserve JHotDraw's existing architecture and extension points.
5. Verify copy, paste, selection after paste, unsupported data handling, text paste, and undo/redo with automated tests.
6. Document what was done, what was not done, and what remains as technical debt.

The selected change is intentionally small. Maintenance work becomes risky when a developer starts fixing adjacent smells only because they are nearby. This report therefore focuses on the paste duplication discovered in

`DefaultDrawingViewTransferHandler` and leaves wider action-layer cleanup as future work.

# Initiation

## Software Change Process Model

The course uses a phased software change process. A change begins with a change request, then moves through concept location, impact analysis, prefactoring, actualization, postfactoring, verification, and conclusion.

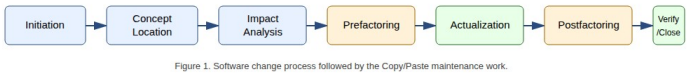


Figure 1. Software change process followed by the Copy/Paste maintenance work.

*Software change process applied to the Copy/Paste work*

In this project, the phases were applied as follows:

| Phase | Applied to this work |
|---|---|
| Initiation | Define the Copy/Paste maintenance change and user story. |
| Concept location | Locate copy/paste classes using search, dependency tracing, and runtime reasoning. |
| Impact analysis | Estimate affected packages/classes and choose a small changed set. |

| Phase | Applied to this work |
| --- | --- |
| Prefactoring | Identify duplicated code, dead code, naming/comment issues, and safe refactoring moves. |
| Actualization | Apply Extract Method in the paste path and clean related small smells. |
| Postfactoring | Review the new structure against SOLID and remaining debt. |
| Verification | Add and run JUnit tests plus BDD-style scenarios. |
| Conclusion | Record the new maintainable state and remaining risks. |

## Change Request

The initiated maintenance request is:

> Maintain the existing JHotDraw Basic Editing Copy/Paste feature by improving its maintainability and verification while preserving the current user-visible behaviour.

The change is preventive maintenance. The feature already exists and should behave the same to users after the change. The benefit is that future changes to paste import behaviour become cheaper and less error-prone.

## User Story

The central user story is:

> As a user editing a drawing, I want to copy selected figures and paste them back into the drawing, so that I can duplicate existing work without recreating it manually.

Related task-level expectations are:

| Requirement | Acceptance criterion |
| --- | --- |
| Copy selected figures | A selected figure can produce transferable drawing data. |
| Empty copy | Copy with no selected figures creates no drawing-figure transferable. |
| Paste supported drawing data | A supported transferable is imported |

| Requirement | Acceptance criterion |
| --- | --- |
| | into the active drawing. |
| Selection after paste | Pasted figures become selected. |
| Undoable paste | Paste creates an undoable edit; undo removes pasted figures and redo restores them. |
| Unsupported data | Unsupported clipboard data is rejected without corrupting the drawing. |
| Extensibility | Registered `InputFormat` and `OutputFormat` implementations continue to decide supported formats. |

### Team Pipeline

The branch follows a GitHub-flow style: the work lives on `copy-paste-basic-editing-labs`, branched from `develop`. The repository contains `.github/workflows/maven.yml`, which runs on push and pull request events targeting `develop`.

The workflow:

1. checks out the repository;
2. sets up Temurin JDK 23 with Maven cache;
3. writes GitHub Packages credentials into Maven settings;
4. runs `mvn -B clean install --file pom.xml`;
5. runs `mvn test --file pom.xml`.

This is a self-testing pipeline for integration into `develop`. However, because the workflow is configured for `develop` events, a feature-branch push alone is not enough evidence of remote CI success. This report therefore claims local Maven verification and describes the CI workflow as available for PR integration, not as already proven green for the feature branch.

## Concept Location

The objective of concept location is to find the work area: where the user-visible Copy/Paste concept is implemented in the code. The search combined static source-code search and dependency tracing.

The first static searches used names from the user story and Java transfer API:

```
copy
paste
clipboard
Transferable
TransferHandler
InputFormat
OutputFormat
```

The initial hits were `CopyAction` and `PasteAction`, but those classes are thin command entry points. Following their calls led into Swing's `TransferHandler` mechanism, and from there to `DefaultDrawingViewTransferHandler`, where selected figures are exported and imported figures are added back to a drawing.

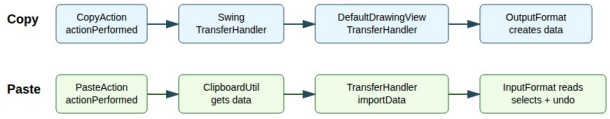


Figure 2. Runtime path located for Copy/Paste.

*Runtime path from actions to transfer handler*

## Static Search and Dependency Search

Grep/text search and dependency search play different roles:

| Technique | Strength | Weakness | Use in this project |
|---|---|---|---|
| Static grep/text search | Fast entry into unfamiliar code. | Finds false positives and misses concepts named | Found `CopyAction`, `PasteAction`, `ClipboardUtil`, |

| Technique | Strength | Weakness | Use in this project |
|---|---|---|---|
| | | differently. | transfer classes. |
| Dependency search | Follows real program relationships. | Requires a good starting point. | Followed actions into `TransferHandler`, `DefaultDrawingView`, `InputFormat`, and `OutputFormat`. |
| Dynamic reasoning/debugger path | Shows which code path is executed for a scenario. | Only covers the exercised scenario. | Used to reason about copy/paste runtime flow and decide where tests should exercise the path. |

## Domain Classes and Responsibilities

| Domain class | Responsibility |
|---|---|
| `CopyAction` | User command entry point for Copy. Resolves the focused component and delegates to `TransferHandler.exportToClipboard`. |
| `PasteAction` | User command entry point for Paste. Reads clipboard contents through `ClipboardUtil` and delegates to `TransferHandler.importData`. |
| `CutAction` | Adjacent clipboard command using the same export mechanism with move semantics. |
| `DuplicateAction` | Related edit command; useful for understanding basic-editing grouping, but not part of the chosen change. |
| `AbstractSelectionAction` | Common base for selection-related edit actions and action enablement. |
| `EditableComponent` | UI contract for editable components. |
| `ClipboardUtil` | Provides clipboard access, with fallback behaviour. |

| Domain class | Responsibility |
|---|---|
| `AWTClipboard` and `OSXClipboard` | Platform-specific clipboard wrappers. |
| `CompositeTransferable` | Combines several transfer flavours into one clipboard payload. |
| `DefaultDrawingView` | Drawing canvas component; owns selection state and installs `DefaultDrawingViewTransferHandler`. |
| `DrawingView` | Abstraction used by the transfer handler. |
| `DefaultDrawingViewTransferHandler` | Core Copy/Paste coordinator: exports selected figures, imports supported transfer data, selects imported figures, moves dropped figures, and fires undo edits. |
| `Drawing` and `AbstractDrawing` | Drawing model and registry for input/output formats. |
| `Figure` | Domain object being copied, pasted, selected, transformed, and serialized. |
| `InputFormat` | Strategy interface for paste/import. |
| `OutputFormat` | Strategy interface for copy/export. |
| `DOMStorableInputOutputFormat` | Native JHotDraw DOM/XML transfer format used by the Draw sample. |
| `ImageInputFormat` and `ImageOutputFormat` | Support image transfer. |
| `TextInputFormat` | Supports text paste when registered on the drawing. |
| `DrawView` and `DrawingPanel` | Sample application setup that registers formats and exposes the editor actions. |

## Concept Location Conclusion

Copy/Paste is not implemented as one method named `copyFigure()` or `pasteFigure()`. It is a framework collaboration. The actions are controllers, the clipboard classes are infrastructure, the formats are strategies, and the drawing/figure classes are domain objects.

The class most relevant to the selected maintenance change is `DefaultDrawingViewTransferHandler`. It is the first class where the copy/paste concept becomes concrete enough to contain both the smell and the fix.

## Impact Analysis

The objective of impact analysis is to predict which parts of the system are affected before changing code. This reduces accidental scope growth and helps decide where verification is needed.

### Static Impact Analysis

The initial impact set from concept location was:

| Class | Reason |
|---|---|
| `CopyAction` | User command entry point for copy. |
| `PasteAction` | User command entry point for paste. |
| `CutAction` | Shares clipboard/export behaviour with copy. |
| `DefaultDrawingViewTransferHandler` | Core import/export behaviour and actual smell location. |
| `DefaultDrawingView` | Installs the transfer handler and owns selection. |
| `Drawing` | Supplies figures and registered formats. |
| `InputFormat` / `OutputFormat` | Define supported import/export strategies. |
| `DOMStorableInputOutputFormat` | Main drawing transferable format in the sample. |
| `ClipboardUtil` / `CompositeTransferable` | Clipboard access and multi-flavour transfer payload. |

The important static observation is that the public contracts could stay unchanged. The handler already depends on abstractions such as `DrawingView`, `Drawing`, and `InputFormat`. Therefore, changing private internals inside the handler should not require changes to action IDs, menu setup, clipboard APIs, file formats, or drawing interfaces.

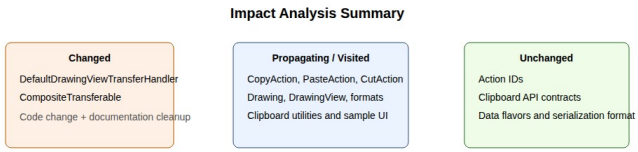


Figure 3. Many packages were inspected, but the actual changed set stayed intentionally small.

*Impact summary: many classes visited, few changed*

## Estimated Changed Set

Before changing code, the estimated changed set was:

| File | Expected change |
|---|---|
| `DefaultDrawingViewTransferHandler.java` | Main refactoring of duplicated paste import and undo code. |
| `CompositeTransferable.java` | Possible documentation/comment cleanup if encountered during transfer-related work. |
| Copy/Cut/Paste action classes | Possible future duplicate-focus lookup cleanup, but intentionally out of scope for this change. |

The actual production changed set matched the conservative estimate:

```
jhotdraw-core/src/main/java/org/jhotdraw/draw/
DefaultDrawingViewTransferHandler.java
jhotdraw-datatransfer/src/main/java/org/jhotdraw/datatransfer/Composit
eTransferable.java
```

Tests were added in:

`jhotdraw-samples/jhotdraw-samples-misc/src/test/java/org/jhotdraw/samples/draw/`

## Dynamic Impact Analysis

Dynamic impact analysis asks whether the modified code is actually part of the runtime path. The refactored method `importTransferData(...)` is called from both the Mac and non-Mac branches of `DefaultDrawingViewTransferHandler.importData(...)` when a registered `InputFormat` supports a transfer flavour.

The automated tests exercise this real handler path by:

1. creating a real `DefaultDrawing`;
2. registering a real `DOMStorableInputOutputFormat` or `TextInputFormat`;
3. using `DefaultDrawingViewTransferHandler.createTransferable(...)` for copy-like behaviour;
4. passing a crafted `Transferable` into `DefaultDrawingViewTransferHandler.importData(...)`;
5. asserting drawing contents, selection, and undo/redo behaviour.

This avoids the operating-system clipboard but still exercises the changed transfer-handler logic.

## Package List

| Package name | # classes visited | Comments |
|---|---|---|
| `org.jhotdraw.action.edit` | 5 | Copy, cut, paste, duplicate, and shared selection action behaviour. Propagating package, unchanged. |
| `org.jhotdraw.api.gui` | 1 | Editable component contract used by basic-editing actions. Unchanged. |
| `org.jhotdraw.draw` | 6 | Drawing view, drawing abstraction, editor/selection, transfer handler. Contains the changed |

| Package name | # classes visited | Comments |
|---|---|---|
| | | class. |
| `org.jhotdraw.draw.figure` | 3 | Figure abstractions and concrete figures created by paste formats. Unchanged. |
| `org.jhotdraw.draw.io` | 7 | Input/output format interfaces and implementations. Propagating package, unchanged. |
| `org.jhotdraw.datatransfer` | 7 | Clipboard proxies and transferable wrappers. `CompositeTransferable` received small cleanup. |
| `org.jhotdraw.app` | 2 | Menu registration and application integration. Unchanged. |
| `org.jhotdraw.gui.action` | 1 | Toolbar/popup action collections. Unchanged. |
| `org.jhotdraw.samples.draw` | 3 | Draw sample setup and test fixtures. Production code unchanged; tests added here. |

Impact analysis conclusion: the feature is scattered for comprehension, but the required code change is localized. This is exactly the kind of case where concept location is larger than the final diff.

## Prefactoring

Prefactoring prepares code before or during a change so the actual maintenance step is small and controlled. In this project, the maintenance change is itself a

behaviour-preserving refactoring, so Clean Code and refactoring concepts were applied directly to the work area.

## Code Smells Found

The main smell was Duplicated Code in `DefaultDrawingViewTransferHandler.importData(...)`. The method contained separate search loops for Mac OS X and non-Mac platforms because of data-flavour ordering differences. That platform-specific search difference is valid, but once a supported `InputFormat` was found, both branches repeated the same successful-import sequence:

1. snapshot existing figures;
2. read the transferable into the drawing;
3. compute imported figures;
4. clear the old selection;
5. select imported figures;
6. add imported figures to the transfer set;
7. move them to the drop point if needed;
8. create and fire an undoable paste edit;
9. return success, or try another format after an `IOException`.

The undoable paste edit construction was also repeated in the file-list import branch.

Other smells found:

| Smell | Location | Resolution |
|---|---|---|
| Duplicated Code | Successful paste import block repeated in `importData(...)`. | Extracted `importTransferData(...)`. |
| Duplicated undo construction | Anonymous paste undo edit repeated. | Extracted `firePasteUndoableEdit(...)`. |
| Dead Code | Private `getDrawing()` method throwing `UnsupportedOperationException`. | Removed. |
| Misleading/noisy comments | “This ugly code sequence…” repaint | Rewritten as intent-explaining comment. |

| Smell | Location | Resolution |
| --- | --- | --- |
| | comment. | |
| Javadoc typos | `CompositeTransferable` documentation. | Corrected class and return-description spelling. |
| Mutable field declaration smell | `CompositeTransferable` collections never reassigned. | Declared fields `final`. |

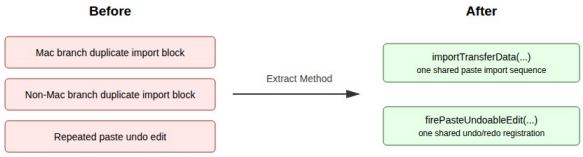


Figure 4. The duplicated paste code was replaced by named helper methods.

*Refactoring before and after*

## Meaningful Names

Clean Code emphasizes names that reveal intent. The extracted method names state the purpose of the hidden detail:

| Method | Meaning |
| --- | --- |
| `importTransferData(...)` | Imports one supported transferable through one selected `InputFormat`. |
| `firePasteUndoableEdit(...)` | Creates and fires the undoable edit for a paste operation. |

After the extraction, the high-level search loop reads as:

```
if (format.isDataFlavorSupported(flavor)) {
    if (importTransferData(comp, t, transferFigures, dropPoint, view,
drawing, format)) {
        retValue = true;
        break SearchLoop;
    }
}
```

That is more intention-revealing than repeating the entire import transaction in each search branch.

## Functions and Stepdown Rule

The Stepdown Rule says code should read top-down: high-level functions should call lower-level functions, which then contain the detail. The refactoring improves `importData(...)` by keeping it responsible for format negotiation while lower-level helper methods perform the import transaction and undo registration.

The extracted import method now contains the detailed transaction:

```
private boolean importTransferData(
        final JComponent comp,
        Transferable t,
        final HashSet<Figure> transferFigures,
        final Point dropPoint,
        final DrawingView view,
        final Drawing drawing,
        InputFormat format) throws UnsupportedFlavorException {
    LinkedList<Figure> existingFigures = new
LinkedList<>(drawing.getChildren());
    try {
        format.read(t, drawing, false);
        final LinkedList<Figure> importedFigures = new
LinkedList<>(drawing.getChildren());
        importedFigures.removeAll(existingFigures);
        view.clearSelection();
        view.addToSelection(importedFigures);
        transferFigures.addAll(importedFigures);
        moveToDropPoint(comp, transferFigures, dropPoint);
        firePasteUndoableEdit(drawing, importedFigures);
        return true;
    } catch (IOException e) {
        e.printStackTrace();
        // Failed to read transferable; try with the next InputFormat.
        return false;
    }
}
```

The method has a remaining weakness: it has seven parameters. That is documented as residual debt and a possible future `ImportContext` parameter-object refactoring.

### Comments and Formatting

The refactoring keeps comments where they explain intent:

```
// Failed to read transferable; try with the next InputFormat.
```

and:

```
// Ensure that the drawing view repaints the area containing the
dropped figures.
```

This is better than apologizing for the code. The extracted methods also improve vertical openness because logical blocks are separated by method boundaries, and improve vertical closeness because related operations are kept together inside one method.

### Prefactoring Summary

The prefactoring result is a smaller and clearer change target:

```
Before:
importData(...) contains repeated successful-import logic in multiple
branches.

After:
importData(...) negotiates formats.
importTransferData(...) performs the successful import transaction.
firePasteUndoableEdit(...) owns undo/redo registration for paste.
```

## Actualization

The objective of actualization is to incorporate the planned change into the existing system. Since this is preventive maintenance, the user-visible Copy/Paste behaviour should remain unchanged. The actualized change preserves the old architecture:

| Area | After actualization |
|---|---|
| `CopyAction` and `PasteAction` | Still delegate to Swing `TransferHandler`. |
| `DefaultDrawingView` | Still installs |

| Area | After actualization |
| --- | --- |
| `DefaultDrawingViewTransferHandler` | `DefaultDrawingViewTransferHandler`. Still coordinates drawing transfer import/export. |
| `InputFormat` / `OutputFormat` | Still define supported data formats. |
| `Drawing` / `Figure` | Still represent the drawing model and copied/pasted objects. |
| Clipboard infrastructure | Still uses `ClipboardUtil` and `CompositeTransferable`. |

## Changed Production Files

| File | Change | Purpose |
| --- | --- | --- |
| `DefaultDrawingViewTransferHandler.java` | Extracted `importTransferData(...)`. | Remove duplicated paste import transaction. |
| `DefaultDrawingViewTransferHandler.java` | Extracted `firePasteUndoableEdit(...)`. | Centralize paste undo/redo registration. |
| `DefaultDrawingViewTransferHandler.java` | Removed unused `getDrawing()`. | Remove dead code. |
| `DefaultDrawingViewTransferHandler.java` | Improved comments around failed import and repaint behaviour. | Improve reader comprehension. |
| `CompositeTransferable.java` | Fixed Javadoc typos and made collections `final`. | Reduce documentation noise and clarify immutable field references. |

## SOLID Principles in the Result

### Single Responsibility Principle

At class level, `DefaultDrawingViewTransferHandler` is still an integration class. It coordinates several responsibilities because it sits at the boundary of Swing transfer, drawing state, selection, and undo. However, the refactoring improves SRP at method level:

| Responsibility | Location after refactoring |
|---|---|
| Search for compatible formats | `importData(...)` |
| Import one supported transferable | `importTransferData(...)` |
| Move transferred figures | `moveToDropPoint(...)` |
| Fire paste undo edit | `firePasteUndoableEdit(...)` |

### Open-Closed Principle

The paste mechanism remains open for extension through `InputFormat`. New clipboard formats can be supported by implementing/registering a new `InputFormat`; the public transfer-handler API does not need to change.

The BDD-style text-paste scenario demonstrates this property: when a drawing registers `TextInputFormat`, a plain-text transferable can produce a text-holder figure without changing the handler.

### Liskov Substitution Principle

`DefaultDrawingViewTransferHandler` treats `InputFormat` implementations uniformly through `isDataFlavorSupported(...)` and `read(...)`. It does not downcast to concrete formats. `DOMStorableInputOutputFormat`, `ImageInputFormat`, and `TextInputFormat` remain substitutable through the same interface.

### Interface Segregation Principle

JHotDraw separates import and export into `InputFormat` and `OutputFormat`. Copy only needs output behaviour; paste only needs input behaviour. A format can implement one or both according to its role.

### Dependency Inversion Principle

The high-level transfer policy depends on abstractions (`Drawing`, `DrawingView`, `InputFormat`, `OutputFormat`) instead of concrete format classes. The concrete details are supplied by the drawing setup. This dependency direction is why the refactoring could stay local.

## Actualization Conclusion

The performed change was behaviour-preserving by design. It did not change action IDs, menu wiring, shortcut wiring, clipboard APIs, `InputFormat`,

`OutputFormat`, or drawing model contracts. The main change is internal structure: one repeated paste transaction became one named helper method, and paste undo creation now has one named home.

## Postfactoring

Postfactoring reviews the structure after actualization and identifies what remains.

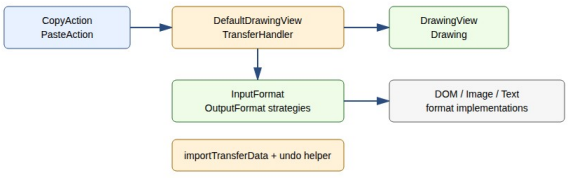


Figure 5. Postfactoring structure: actions stay thin, formats stay strategies, and paste internals have clearer boundaries.

*Postfactoring structure*

## Before and After Structure

Before the change:

```
DefaultDrawingViewTransferHandler.importData(...)
  Mac search branch
    repeated successful import block
  non-Mac search branch
    repeated successful import block
  file-list import branch
    repeated paste undo edit construction
```

After the change:

```
DefaultDrawingViewTransferHandler.importData(...)
  Mac search branch
    importTransferData(...)
  non-Mac search branch
    importTransferData(...)
  file-list import branch
    firePasteUndoableEdit(...)

importTransferData(...)
  read transferable
  compute imported figures
  update selection
  move figures if needed
  fire undoable paste edit

firePasteUndoableEdit(...)
  undo removes imported figures
  redo adds imported figures again
```

## Maintainability Improvement

The key maintainability improvement is that future paste fixes now have fewer places to change. Before the refactor, a future developer changing selection-after-paste or paste undo behaviour could easily update one branch but forget another. After the refactor, a successful import transaction is represented by `importTransferData(...)`, and paste undo behaviour is represented by `firePasteUndoableEdit(...)`.

## Remaining Debt

The refactoring does not make the transfer handler perfect. Remaining debt includes:

| Remaining issue | Risk | Suggested future work |
|---|---|---|
| `DefaultDrawingViewTransferHandler` is still large. | New maintainers must understand several transfer concerns at once. | Extract format-search or file-list import collaborators. |
| `importTransferData(...)` has seven parameters. | Helper method is slightly parameter-heavy. | Introduce an `ImportContext` value object if the method grows. |
| Copy/Cut/Paste actions share similar focus- | Duplicated action-layer | Refactor shared lookup into the common action |

| Remaining issue | Risk | Suggested future work |
|---|---|---|
| owner lookup. | code remains. | base class after adding tests. |
| Cut path needs more explicit tests. | Removing cut-related duplication would be risky without characterization tests. | Add cut + undo/redo tests before touching `exportDone(...)`. |
| Error handling still uses `printStackTrace()` in places. | Console-only failure reporting is weak. | Introduce consistent logging/error policy. |

Postfactoring conclusion: the selected smell was reduced without widening the public interface. The class still contains broader technical debt, but the change stays aligned with the selected user story and impact analysis.

## Verification

Verification confirms that the changed behaviour still satisfies the user story. The most important code under test is the paste import path through `DefaultDrawingViewTransferHandler` and the DOM transferable round-trip used by copy/paste.

### Test Files

| Test file | Tests | Purpose |
|---|---|---|
| `DOMStorableInputOutputFormatTest.java` | 3 | Tests the native DOM transferable round-trip and unsupported flavour rejection. |
| `DefaultDrawingViewTransferHandlerTest.java` | 5 | Tests selected copy, empty copy, supported paste, selection after paste, unsupported paste, and undo/redo. |
| `CopyPasteBddTest.java` | 5 | Encodes user-story scenarios in Given-When-Then style using JUnit 4. |

The test files are in:

`jhotdraw-samples/jhotdraw-samples-misc/src/test/java/org/jhotdraw/samples/draw/`

## Unit and Component-Level Tests

The tests use real JHotDraw objects:

| Collaborator | Why used |
|---|---|
| `DefaultDrawing` | Real drawing model. |
| `DefaultDrawingView` | Real drawing view and selection state. |
| `DefaultDrawingViewTransferHandler` | Real class under test. |
| `DOMStorableInputOutputFormat` | Real native JHotDraw copy/paste format. |
| `DrawFigureFactory` | Real sample factory needed for DOM figure creation. |
| `RectangleFigure` / `TextFigure` | Concrete figure types used in realistic fixtures. |
| `UndoManager` | Real Swing undo manager used to test paste undo/redo. |

The tests avoid the real operating-system clipboard. Instead, they keep a `Transferable` returned by the copy path and pass it directly into `importData(...)`. This is a test seam, not a change in production behaviour. It makes the suite deterministic and headless-safe.

## Test Coverage Against Acceptance Criteria

| Acceptance criterion | Automated evidence |
|---|---|
| Copy creates transfer data from selected figures. | `copySelectedFigureCreatesTransferable` |
| Copy with empty selection creates no drawing figure transferable. | `copyWithoutSelectionCreatesNoTransferable` |
| Supported paste imports a distinct figure. | `pasteSupportedTransferableAddsAndSelectsImportedFigure` |
| Pasted figures become selected. | `pasteSupportedTransferableAddsAndSelectsImportedFigure` and BDD scenario 1 |
| Paste fires undoable edit. | `pasteFiresUndoableEditThatCanUndo` |

| Acceptance criterion | Automated evidence |
| --- | --- |
| | `AndRedoImportedFigure` |
| Unsupported data leaves drawing unchanged. | `rejectsUnsupportedDataFlavor`, `pasteUnsupportedTransferableLeavesDrawingUnchanged`, BDD scenario 3 |
| Registered formats drive supported content. | DOM round-trip tests and text-paste BDD scenario |

## BDD Test and Scenario Analysis

The BDD lab discusses JGiven. JGiven is a Java BDD framework that structures tests as Given/When/Then stages and can generate readable scenario reports. This branch does not add JGiven as a dependency. Instead, the project uses JUnit 4 test methods whose names and arrange/act/assert structure express Given/When/Then scenarios. This keeps the dependency footprint small while still connecting tests back to the user story.

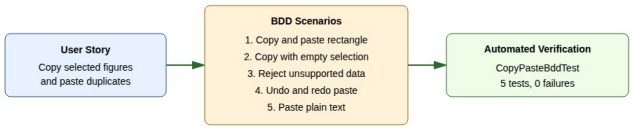


Figure 6. The user story is mapped to executable Given-When-Then scenarios.

*BDD verification*

| User-story aspect | BDD scenario | Test method |
| --- | --- | --- |
| Copy and paste selected object | Given a drawing with one selected rectangle, when the user copies | `givenDrawingWithOneSelectedRectangle_whenUserCopiesAndPastesIt_thenDrawingContainsTw` |

| User-story aspect | BDD scenario | Test method |
|---|---|---|
| | and pastes it, then the drawing contains two rectangles and the pasted rectangle is selected. | `oRectanglesAndPastedRectangleIsSelected` |
| Copy requires selection | Given no selected figures, when the user invokes copy, then no drawing-figure transferable is produced. | `givenDrawingViewWithNoSelectedFigures_whenUserInvokesCopy_thenNoDrawingFigureTransferableIsProduced` |
| Unsupported data is safe | Given unsupported clipboard data, when the user invokes paste, then the drawing remains unchanged. | `givenClipboardContainsUnsupportedData_whenUserInvokesPaste_thenDrawingRemainsUnchanged` |
| Paste is undoable | Given pasted figures in a drawing, when undo and redo are invoked, then pasted figures are removed and added back. | `givenUserPastedFiguresIntoDrawing_whenUserInvokesUndoAndRedo_thenPastedFiguresAreRemovedAndAddedBack` |
| Other supported content | Given plain text and a registered `TextInputFormat`, when paste is invoked, then a text-holder figure is added. | `givenClipboardContainsPlainTextAndDrawingRegistersTextInputFormat_whenUserInvokesPaste_thenTextHolderFigureIsAdded` |

## Verification Commands and Results

The sample-module verification command:

```
mvn -pl jhotdraw-samples/jhotdraw-samples-misc test
```

Result from local verification on 11 June 2026:

```
Running
org.jhotdraw.samples.draw.DefaultDrawingViewTransferHandlerTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

```
Running org.jhotdraw.samples.draw.CopyPasteBddTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

Running org.jhotdraw.samples.draw.DOMStorableInputOutputFormatTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

Results:
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

The full reactor command:

```
mvn test --file pom.xml
```

also completed locally with `BUILD SUCCESS`.

## CI Verification Status

The repository contains a GitHub Actions workflow at `.github/workflows/maven.yml`. It runs for push and pull-request events targeting `develop`. It should execute the added Copy/Paste tests when this branch is opened as a pull request against `develop` or merged through that route.

No remote GitHub Actions success result for commit `b4eea564` was available from the local evidence checked during this review. Therefore, the accurate claim is:

> The tests pass locally, and the repository has a CI workflow that will run them on `develop` push/PR events.

The report should not claim:

> The feature branch has already passed GitHub Actions, or the branch has already been merged into `develop`.

unless a real PR/check link or screenshot is added.

## Verification Limits

The automated tests do not exercise the full GUI path through menu selection, keyboard focus, system clipboard ownership, and real operating-system clipboard contents. This is intentional for deterministic CI-friendly tests.

A final manual GUI checklist should be added to the appendix if screenshots are required:

| Manual check | Expected result |
|---|---|
| Start the Draw sample. | Application opens. |
| Create and select a rectangle. | Rectangle appears selected. |
| Invoke Copy through menu or shortcut. | Drawing remains unchanged. |
| Invoke Paste. | Duplicate figure appears. |
| Check selection after paste. | Pasted figure is selected. |
| Invoke Undo. | Pasted figure is removed. |
| Invoke Redo. | Pasted figure is restored. |
| Paste unsupported external content in a DOM-only drawing. | Drawing remains unchanged. |

This manual checklist is not claimed as completed unless screenshots or notes from an actual run are added.

## Conclusion

This report documents a complete maintenance pass over JHotDraw's Basic Editing Copy/Paste feature. The work began with a user story and followed the phased software change process. Concept location showed that Copy/Paste is a collaboration between edit actions, Swing transfer handling, clipboard utilities, drawing views, drawing formats, and the drawing/figure model. The key implementation hotspot is `DefaultDrawingViewTransferHandler`.

Impact analysis showed that many classes must be understood, but few production files need to change. The main code smell was duplicated paste import logic in `DefaultDrawingViewTransferHandler.importData(...)`, especially across the Mac and non-Mac data-flavour search branches. Actualization removed that duplication with Extract Method, producing `importTransferData(...)` and `firePasteUndoableEdit(...)`, removing dead code, improving comments, and preserving public behaviour.

The resulting structure is more maintainable. The transfer handler remains a large integration class, but the repeated import transaction and undo construction now have named locations. This reduces the risk that future changes to paste behaviour diverge between branches.

Verification was strengthened with 13 JUnit 4 tests, including 5 BDD-style scenarios derived from the user story. Local Maven verification passed for both

the sample module and the full reactor. The repository also contains a GitHub Actions workflow for `develop` push/PR events, but remote CI success for this feature branch should only be claimed when a real PR/check result is available.

## Discussion

### What Could Have Been Better

First, the change could have included an `ImportContext` parameter object. `importTransferData(...)` is clearer than the duplicated block, but seven parameters is still a smell. I did not include that extra abstraction because the change was already large enough for one focused maintenance step.

Second, the action layer still contains duplication. `CopyAction`, `CutAction`, and `PasteAction` share similar focused-component lookup logic. Impact analysis found this, but it was left out to avoid widening the changed set. It belongs in a follow-up change with its own tests.

Third, the cut path is not tested as strongly as paste. Since this report focuses on copy/paste and the changed code is in paste import and undo handling, the current tests are acceptable for this scope. Before refactoring cut-specific behaviour, a cut + undo/redo characterization test should be added.

Fourth, the full GUI path remains manual. The automated tests are deterministic because they avoid the OS clipboard. That is good for CI, but a user-facing release should also include screenshots or manual evidence for menu actions, shortcuts, focus, and real clipboard behaviour.

### What Failed and How It Was Mitigated

The main testing risk was the operating-system clipboard. Tests that depend on a real clipboard can fail because the clipboard is global, environment-dependent, and often unavailable or unreliable in headless CI. The mitigation was to pass crafted `Transferable` objects directly to `DefaultDrawingViewTransferHandler.importData(...)`. This still exercises the changed import logic while avoiding environment flakiness.

Another risk was documentation overclaiming. Some draft reports claimed that the branch was already merged into `develop` or that GitHub Actions had passed for the feature branch. The local repository does not prove those claims. This

merged report fixes that by claiming only what is evidenced: local Maven tests pass, and the workflow is configured for `develop` push/PR events.

### Lessons Learned

The main lesson is that impact analysis protects scope. JHotDraw contains many adjacent smells, but a good maintenance report should not fix everything at once. The safe path was to locate the feature, identify the core duplicated paste logic, change only the relevant production files, add tests for the changed behaviour, and record remaining debt for future work.

## References and Sources


[1] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

[2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[3] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.

[4] M. Fowler, "Continuous Integration," 2006. Available: https://martinfowler.com/articles/continuousIntegration.html

[5] D. North, "Introducing BDD," 2006. Available: https://dannorth.net/introducing-bdd/

[6] JGiven, "BDD in plain Java." Available: https://jgiven.org

[7] W. Randelshofer, JHotDraw 7 project material and source code.

[8] Team fork repository: `https://github.com/JakubPotocky/JHotDraw`, branch `copy-paste-basic-editing-labs`, commit `b4eea56487ea289d6c1855d9ba32150e16b31366`, inspected 11 June 2026.

[9] SDU Software Maintenance course material: Intro Lab, Change Request Lab, Concept Location Lab, Analysis Lab, Refactoring Lab, Actualization Lab, Testing Lab, BDD Lab, and Continuous Integration Lab.

# Appendix

## Appendix A - Changed Production Files

```
jhotdraw-core/src/main/java/org/jhotdraw/draw/
DefaultDrawingViewTransferHandler.java
jhotdraw-datatransfer/src/main/java/org/jhotdraw/datatransfer/Composit
eTransferable.java
```

## Appendix B - Added Test Files

```
jhotdraw-samples/jhotdraw-samples-misc/src/test/java/org/jhotdraw/
samples/draw/DOMStorableInputOutputFormatTest.java
jhotdraw-samples/jhotdraw-samples-misc/src/test/java/org/jhotdraw/
samples/draw/DefaultDrawingViewTransferHandlerTest.java
jhotdraw-samples/jhotdraw-samples-misc/src/test/java/org/jhotdraw/
samples/draw/CopyPasteBddTest.java
```

## Appendix C - Main Git Evidence

```
Branch: copy-paste-basic-editing-labs
Commit: b4eea56487ea289d6c1855d9ba32150e16b31366
Commit message: Copy/paste labs with tests and docs
```

The local branch is not an ancestor of `origin/develop` at inspection time, so this report does not claim it has been merged into `develop`.

## Appendix D - Local Verification Commands

```
mvn -pl jhotdraw-samples/jhotdraw-samples-misc test
mvn test --file pom.xml
```

Both commands completed locally with `BUILD SUCCESS` during the review on 11 June 2026.

## Appendix E - Figure List

| Figure | Source file |
|---|---|
| Software change process | `bob/docs/portfolio/figures/png/software-change-process.png` |
| Copy/Paste runtime path | `bob/docs/portfolio/figures/png/copy-paste-runtime-path.png` |
| Impact summary | `bob/docs/portfolio/figures/png/impact-summary.png` |
| Refactoring before/after | `bob/docs/portfolio/figures/png/refactoring-before-after.png` |
| Postfactoring structure | `bob/docs/portfolio/figures/png/postfactoring-structure.png` |
| BDD verification | `bob/docs/portfolio/figures/png/bdd-verification.png` |

## Appendix F - Material Not Claimed

The following claims should only be added if concrete evidence is attached:

| Possible evidence | Current status |
|---|---|
| Pull request URL for `copy-paste-basic-editing-labs` into `develop`. | Not available from local evidence. |
| Remote GitHub Actions success for commit `b4eea564`. | Not available from local evidence. |
| JGiven-generated scenario report. | Not present; BDD scenarios are JUnit 4 Given-When-Then style. |
| Manual GUI screenshots using real menus and OS clipboard. | Not present in this generated report. |
| Merge commit into `develop`. | Not present in the local branch graph at inspection time. |